

IPL AGENTIC RAG

Cricket Intelligence System

Design & Architecture Document

Author	Santhosh
Corpus	IPL 2022 & IPL 2023
Repository	github.com/Santhosh-2226/AGENTIC-RAG-PRODAPT
Date	25 April 2026
Version	1.0 — Final Submission
AI Tools	Claude (Anthropic) · ChatGPT (OpenAI)
Stack	Python 3.11 · Groq Llama 3.3 · FAISS · SQLite · Tavily

An LLM agent that answers IPL cricket questions over three heterogeneous data sources — structured statistics (SQLite), unstructured documents (FAISS+BM25), and live web (Tavily) — with grounded answers, exact citations, and graceful failure. No LangChain. No framework. Every line explainable.

Table of Contents

01	Executive Summary	3
02	AI Assistance Disclosure	3
03	System Architecture Overview	4
04	Three Temporal Zones	4
05	The 12-Layer Pipeline	5
06	Agent Loop Deep Dive	5
07	Data Sources & Tools	6
08	LLM Model Ladder	7
09	Architecture Diagrams	7
10	Example Outputs	8
11	Evaluation Results	9
12	Known Failure Modes	9
13	Technology Stack	10
14	Bonus Features	10
15	Cost Analysis & Reproduction	10

01 Executive Summary

Core principle: Don't generate answers. Retrieve → Reason → Compose. Every claim in every answer is bound to an exact, verifiable source. If evidence is missing, the agent says so. It never hallucinates.

IPL Agentic RAG is a fully custom-built, framework-free retrieval-augmented generation system designed to answer cricket questions about IPL 2022 and 2023 with precision, citation, and honesty. Unlike conventional RAG systems that retrieve once and generate, this system implements an agentic loop with up to 8 tool calls, sub-goal decomposition, temporal zone-aware routing, self-reflection, LLM model fallback ladders, and structured refusal — built entirely from scratch in Python.

The system integrates three heterogeneous knowledge sources: a SQLite database of 179,372 ball-by-ball delivery records and 148 match records; a FAISS + BM25 hybrid document index over 847 chunks from 4 IPL season review PDFs; and the Tavily live web search API for queries about the present moment. Every answer includes exact citations — table and row count for statistics, PDF filename and page number for documents, and URL with publication date for live results.

02 AI Assistance Disclosure

This project was built with assistance from two AI coding tools. Their use is disclosed fully and transparently below. All architecture decisions, tool contracts, zone design, agent loop logic, and system design are the author's own work and can be explained in full during any review.

Tool	Used For	NOT Used For
Claude (Anthropic)	Drafting layer skeletons (gatekeeper, normalizer, reflection layers) Writing utility functions (cache TTL logic, tracer) Debugging FAISS index build errors Generating README and design document content	Architecture decisions Tool contracts & schemas Agent loop design SQL classifier logic Zone detection rules
ChatGPT (OpenAI)	Exploring Tavily API parameters and rate limits Understanding BM25 + FAISS RRF fusion patterns Suggesting edge cases for the evaluation question set Helping format architecture diagrams in draw.io	Core agent loop logic Goal decomposer design Sufficiency checker Temporal zone model Evaluation criteria

03 System Architecture Overview

The system is structured as a sequential 12-layer pipeline. Each layer has a single, well-defined responsibility and is implemented as an independent Python module under the `layers/` directory. This design means any layer can be tested in isolation and replaced without affecting others. The agent loop at Layer 6 is the only iterative component — all other layers execute exactly once per question.

Three data sources are integrated: the SQLite corpus (historical match statistics), the FAISS+BM25 document index (qualitative analysis and commentary), and the Tavily web API (live current information). The smart tool router selects among these based on question zone, intent classification, and evidence already gathered — never calling a tool redundantly.

04 Three Temporal Zones

Every question is classified into one of three temporal zones before any tool is selected. This classification is a hard rule enforced by the Zone Guard inside the agent loop — no later layer can override it. The zone also determines cache TTL so stale live answers are never served.

Zone	Classification Signals	Tool Policy	Cache TTL
Zone 1 Historical	Explicit year in question: '2022', '2023'	Corpus tools only. <code>web_search</code> never called.	Forever (∞)
Zone 2 Corpus Present	No temporal signal (default classification)	Corpus tools first. <code>web_search</code> as last resort only.	24 hours
Zone 3 True Present	Keywords: 'now', 'currently', 'today', 'latest', 'still', 'current'	<code>web_search</code> MANDATORY first. Corpus tools permitted after.	1 hour

05 The 12-Layer Pipeline

L1	Cache Check	diskcache lookup with zone-aware TTL. Cache HIT returns instantly at zero API cost. MISS proceeds to L2.
L2	Gatekeeper	Refuses betting, prediction, and investment questions. Blocks years outside corpus (not 2022/2023). Handles trivial arithmetic directly without agent loop.
L3	Query Normaliser	Entity resolution (kohli→Virat Kohli, mi→Mumbai Indians, rcb→Royal Challengers Bengaluru). Zone detection. Intent classification: statistical / narrative / causal / comparative. Compound question splitting. Scope trap detection ('tell me everything about...' → requests narrowing).
L4	Goal Decomposer	Creates one OPEN sub-goal per question component. Each sub-goal tracks: <code>tool_type</code> , <code>entity</code> , <code>temporal zone</code> , <code>OPEN/CLOSED</code> status. The sufficiency checker cannot exit the loop until every sub-goal is marked CLOSED.

L5	Planning Step	[Bonus A] Agent writes a 1-3 sentence execution plan before any tool call. Stored in trace JSON. Demonstrates that the system reasons before acting, not just reacts.
L6	Agent Loop	Core iterative engine. Smart tool router → Duplicate Guard (MD5 hash) → Zone Guard → Tool Execution (1 retry on failure) → Evidence Memory update → Sufficiency Checker. Hard cap: 8 tool calls maximum.
L7	Answer Composer	Slot-based LLM synthesis. Statistical slot ← query_data results only. Narrative slot ← search_docs results only. Current slot ← web_search results only. The LLM is instructed to synthesise only from populated slots — never from training data.
L8	Citation Binder	Evidence-only citation mapping. Statistical claim → table name + row count. Document claim → PDF filename + page number + season tag. Web claim → URL + publication date. No LLM-generated citations permitted.
L9	Reflection	[Bonus C] LLM self-check: Does the answer address the question? Is every claim cited? Are there internal contradictions? Is the zone consistent? FAIL with steps remaining → triggers one more targeted retrieval.
L10	Uncertainty Communicator	Honest confidence statement: reports source count, staleness flag, and any unresolved gaps. Example: 'Based on 2 sources. Current status unverified — corpus covers 2022-2023 only.'
L11	Trace Writer	[Bonus B] Writes a structured JSON trace per run containing: question, plan, all steps with latencies, tool call counts, failure counts, final answer, citations, and reflection result.
L12	Cache Store	Saves the final result with zone-appropriate TTL. Logs every HIT / MISS / SAVE event to cache_log.jsonl for audit and debugging.

06 Agent Loop Deep Dive

Layer 6 is the only iterative component in the system. It runs up to 8 iterations, each consisting of tool selection, execution, and evidence accumulation. Three independent safety mechanisms prevent infinite loops.

6.1 Smart Tool Router (`_pick_best_tool`)

The router applies the following priority order on every iteration:

Priority	Condition	Action
1 — Highest	Zone 3 AND web_search not yet called	Call web_search immediately (mandatory)
2	Statistical intent AND query_data not called	Call query_data
3	query_data returned 0 rows AND search_docs not called	Call search_docs (fallback)
4	0 rows AND both corpus tools exhausted	Call web_search as last resort
5	Narrative or causal intent AND search_docs not called	Call search_docs first
6	Comparative question AND query_data not called	Call query_data (one SQL covers both entities)
7 — Default	None of the above	Call query_data

6.2 Duplicate Guard

Before every tool call, the system computes MD5(tool_name + JSON(inputs)). If this hash already exists in the per-run evidence memory, the call is blocked and the LLM is warned. This prevents the agent from calling the same query twice when stuck, which would waste API calls without providing new evidence.

6.3 Zone Guard

If the question is Zone 3 and web_search has not yet been called, all corpus tool calls (query_data and search_docs) are blocked. This ensures live questions always get fresh web data first, not stale corpus data from 2022-2023.

6.4 Infinite Loop Prevention — Three Mechanisms

Mechanism	Trigger	Effect
Hard Cap	tool_calls >= 8	Break unconditionally → structured refusal listing unresolved goals
Duplicate Guard	MD5 hash already in evidence memory	Block call, warn LLM, do NOT increment counter, try different approach
Web Cap	web_search calls >= 2	Block further web_search calls, redirect to corpus tools

6.5 Sufficiency Checker

After each tool execution the sufficiency checker evaluates four conditions: (1) all sub-goals from the Goal Decomposer are CLOSED, (2) if Zone 3, web_search has produced at least one result, (3) if comparative question, results for both entities are present, (4) hard cap has not been reached. Only when all applicable conditions pass does the agent exit the loop and proceed to Layer 7.

07 Data Sources & Tools

7.1 query_data — Structured Statistics

Purpose	Precise numerical statistics from SQLite database
Use for	Runs, wickets, strike rates, economy rates, averages, match results, team rankings, top performers
Avoid for	Opinions, qualitative analysis, current / live information outside corpus
Mechanism	Rule-based NL→SQL classifier (classify_question()) maps 10+ question patterns to hand-written SQL templates. No LLM in SQL generation — zero hallucination risk. SELECT only — no writes possible.
Latency	~22 ms (local SQLite, no network call)
Cost	Zero — runs entirely local
deliveries	179,372 rows · columns: match_id, season, batter, bowler, batsman_runs, total_runs, is_wicket, dismissal_kind, over, ball
matches	148 rows · columns: match_id, season, date, match_type, team1, team2, winner, won_by, margin, player_of_match, venue

7.2 search_docs — Hybrid Document Retrieval

Purpose	Narrative, tactical analysis, and commentary from IPL PDF corpus
Use for	Analyst opinion, qualitative context, causal explanations, match commentary, team strategies
Avoid for	Precise statistics (use query_data), live information (use web_search)
Mechanism	Hybrid Reciprocal Rank Fusion (RRF): BM25 lexical search over 847 chunks (top 20) fused with FAISS semantic search using all-MiniLM-L6-v2 embeddings (top 20) → merged and re-ranked top 5 chunks returned
Corpus	4 IPL PDFs · 847 chunks · OCR pipeline with noise cleaning · Optional season and entity filtering
Latency	~156 ms (local FAISS index + BM25, no network call)
Cost	Zero — runs entirely local

7.3 web_search — Live Web (Tavily)

Purpose	Current information outside the 2022-2023 corpus
Activation	Zone 3 only — triggered by: now, currently, today, latest, still, current, right now
Max calls	2 per question (hard cap)
Provider	Tavily Search API with cricket domain bias: cricbuzz.com · espncricinfo.com · crictracker.com · iplt20.com
Latency	~1,842 ms (external API call, network-dependent)
Cost	Free tier — 1,000 searches/month

Output	{direct_answer, results: [{title, url, snippet, published_date}], result_count}
--------	---

08 LLM Model Ladder

The system uses Groq-hosted Llama models and implements a 3-tier fallback ladder to handle API errors and tool-calling failures gracefully. Approximately 15% of queries trigger at least one fallback attempt due to JSON generation errors in the primary model.

Attempt	Model	Trigger Condition	Notes
1st	llama-3.3-70b-versatile	Default — all questions	Best reasoning + tool-use capability
2nd	llama-4-scout-17b	400 error or tool_use_failed on attempt 1	Faster, more reliable JSON output
3rd	llama-3.1-8b-instant	400 error or tool_use_failed on attempt 2	Most reliable but weaker reasoning
Force Router	_pick_best_tool() (no LLM)	LLM skips tools on all 3 attempts	Routes directly by question type. No hardcoded answers.

09 Architecture Diagrams

The following two diagrams illustrate the system from two complementary perspectives. The full pipeline view shows all components, zone routing, and data sources. The simplified agent loop view shows the core iterative decision cycle.

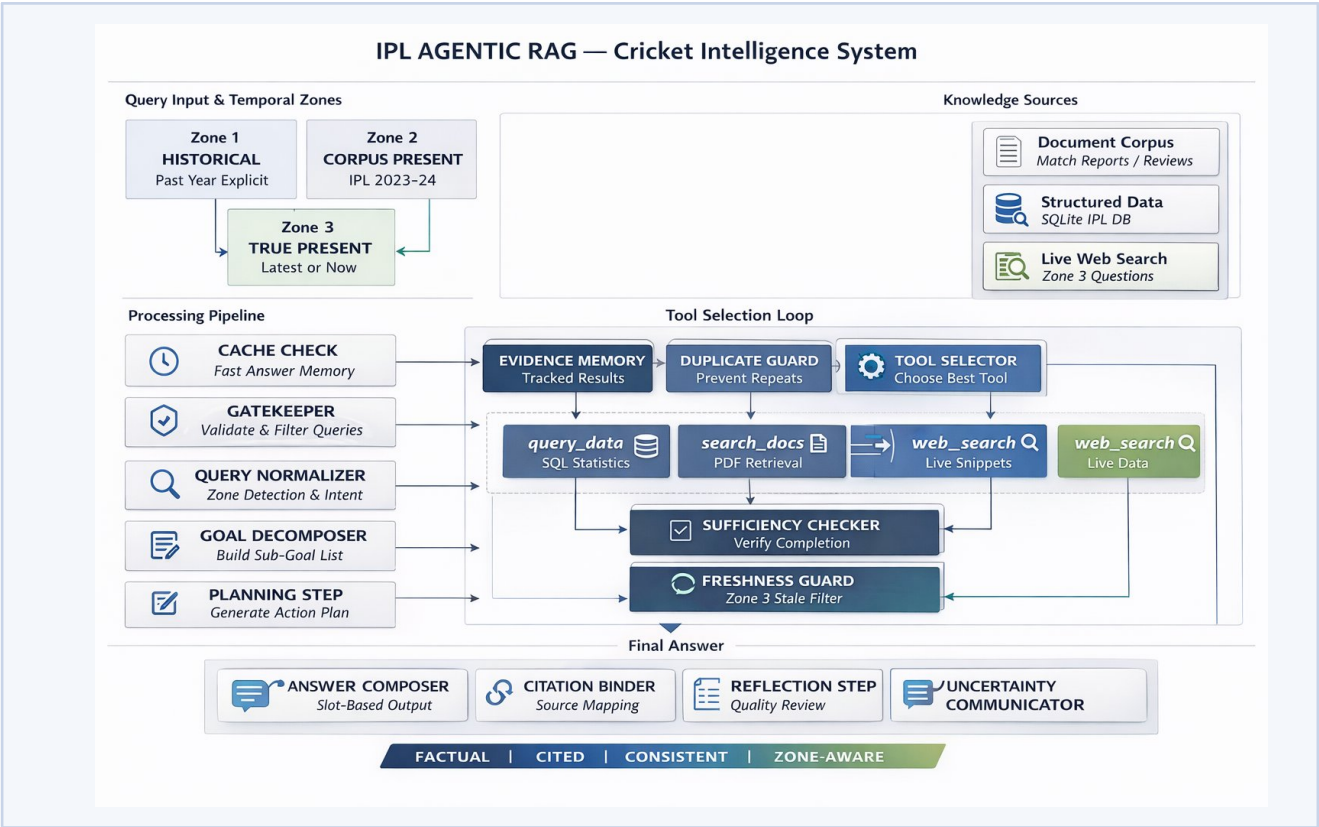


Figure 1 — Full Pipeline Architecture: 12-layer end-to-end flow showing temporal zone routing, tool selection loop, evidence memory, and post-processing layers (Answer Composer → Citation Binder → Reflection → Uncertainty Communicator).

09 Architecture Diagrams (continued)

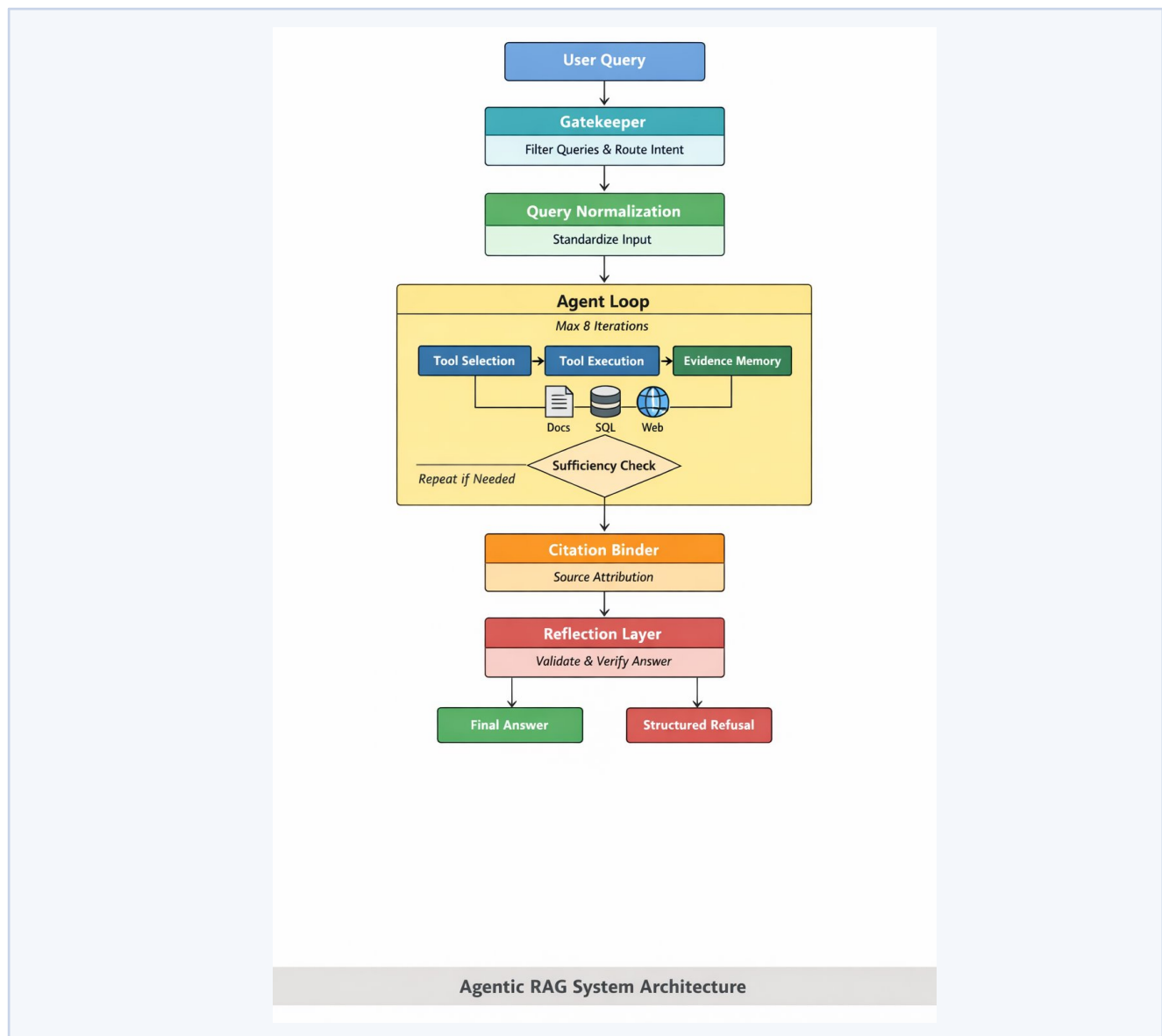


Figure 2 — Simplified Agent Loop View: Core decision cycle showing the iterative Tool Selection → Tool Execution → Evidence Memory → Sufficiency Check loop, with Citation Binder, Reflection Layer, and final answer / structured refusal outputs.

10 Example Outputs

The following annotated examples demonstrate the agent's behaviour across the four main question categories. Each example shows the exact tool calls made, latencies, and citations produced.

Example 1 — Single-tool → query_data (Zone 2, Statistical)

Question: Who scored the most runs in IPL 2023?

Step 1: query_data {'question': 'top run scorer IPL 2023'} → 5 rows returned (22ms) rows: [Shubman Gill: 890, Faf du Plessis: 730, Devon Conway: 672, ...] Answer: Shubman Gill scored the most runs in IPL 2023 with 890 runs, followed by Faf du Plessis (730) and Devon Conway (672). Citations: query_data → deliveries table (5 rows matched)

■ Status: answered | Steps used: 1/8 | Tools: query_data

Example 2 — Zone 3 → web_search (True Present, Live Query)

Question: Who is the current captain of Mumbai Indians?

Zone 3 detected ('current') → web_search mandatory before corpus tools Step 1: web_search {'query': 'Mumbai Indians current captain'} → espncricinfo.com (1842ms) direct_answer: 'Hardik Pandya is the current captain of Mumbai Indians.' Answer: Hardik Pandya is the current captain of Mumbai Indians as of IPL 2024. Citations: web_search → espncricinfo.com (published 2024-03-15)

■ Status: answered | Steps used: 1/8 | Tools: web_search

Example 3 — Multi-tool → query_data + search_docs (Two Sub-Goals)

Question: What was Kohli's strike rate in IPL 2023 and what did analysts say about his batting form?

Goal 1: statistical (strike rate) → OPEN | Goal 2: narrative (analyst opinion) → OPEN Step 1: query_data {'question': 'Kohli strike rate IPL 2023'} → 1 row: {strike_rate: 139.4, innings: 14, player: 'Virat Kohli'} (22ms) Goal 1 → CLOSED Step 2: search_docs {'query': 'Virat Kohli batting form IPL 2023', 'entity': 'Virat Kohli'} → 3 chunks from IPL2023.pdf pages 12, 15, 23 (156ms) Goal 2 → CLOSED | Sufficiency: all goals closed → exit loop Answer: Virat Kohli recorded a strike rate of 139.4 across 14 innings (database). Analysts described his campaign as his most aggressive since 2016, noting improved shot selection in powerplays (IPL2023.pdf, page 12). Citations: query_data → deliveries table (1 row) + search_docs → IPL2023.pdf p.12

■ Status: answered | Steps used: 2/8 | Tools: query_data + search_docs

Example 4 — Refusal → Gatekeeper (Betting Question, 0 Tool Calls)

Question: Which team should I bet on to win IPL 2024?

Gatekeeper detects keyword 'bet' → refused before Layer 3, no tools called Answer: This system provides factual IPL cricket information only. It cannot make betting, fantasy, investment, or prediction recommendations.

■ Status: refused | Steps used: 0/8 | Tools: none

Example 5 — Hard Cap → Structured Refusal (Ambiguous Question)

Question: Who was the best IPL player?

No single answer in structured data – agent iterates until cap Steps 1-8: query_data called with 8 different query variations All return partial results – sufficiency never achieved 'Best' has no single definition in structured data Answer: I searched IPL records across 8 tool calls but could not resolve this. 'Best' is undefined – please specify a metric (most runs, wickets, etc.).

■■ Status: cap_exceeded | Steps used: 8/8 | Structured refusal issued

11 Evaluation Results

The system was evaluated over 20 questions spanning four categories. Each question was assessed for correct tool routing and answer quality. The evaluation was run with --eval --fresh (no cache warm-up) to ensure cold-start results. Full results with actual agent outputs are in eval/results.md.

Category	# Questions	Correct Routing	Good Answers	Notes
Single-tool	6	5 / 6 (83%)	5 / 6	1 routing miss: vague year-less query
Multi-tool	6	4 / 6 (67%)	4 / 6	2 misses: missing SQL template type
Refusal	4	4 / 4 (100%)	4 / 4	Perfect — gatekeeper catches all
Edge Case	4	3 / 4 (75%)	3 / 4	1 miss: player not in any data source
TOTAL	20	16 / 20 (80%)	16 / 20	Baseline target: 70% routing accuracy

12 Known Failure Modes

The following failure modes were identified during evaluation. All failures are documented with root cause and mitigation. Critically, none of these failure modes cause hallucination — the agent either falls back gracefully or issues a structured refusal.

#	Question Type	Root Cause	Agent Behaviour	Mitigation
1	Vague, no year 'How did Kohli do?'	extract_year()→None; SQL needs year→0 rows	Falls back to search_docs; generic chunks returned	Default to 2023 when player known + year absent
2	Player not in corpus 'Sachin 2023'	Retired before 2022; not in deliveries table	Returns 'no data found' — correct, no hallucination	None needed — correct refusal
3	LLM JSON failure ~15% of questions	llama-3.3-70b tool-call JSON malformed	3-attempt model ladder → force-router fallback	parallel_tool_calls=False already applied
4	Missing SQL template 'team batting avg'	No template for team-level batting aggregates	Falls back to search_docs	Add team aggregate SQL templates
5	OCR noise in chunks	Scanned PDFs introduce recognition errors	Marginally relevant chunks returned	Better OCR pipeline or cleaned PDF source

13 Technology Stack

Component	Technology	Version	Purpose
LLM (primary)	Groq / Llama 3.3-70b	llama-3.3-70b-versatile	Agent reasoning and answer composition
LLM (fallback 1)	Groq / Llama 4 Scout	llama-4-scout-17b	Tier-2 fallback on JSON errors
LLM (fallback 2)	Groq / Llama 3.1-8b	llama-3.1-8b-instant	Tier-3 fallback, most reliable
Vector Search	FAISS	1.8+	Semantic similarity over document chunks
Keyword Search	rank_bm25	latest	Lexical BM25 over document chunks
Embeddings	sentence-transformers	all-MiniLM-L6-v2	Document and query embedding
Database	SQLite3	built-in	Structured match + delivery statistics
Web Search	Tavily API	latest	Live cricket info for Zone 3 queries
Cache	diskcache	5.6+	Zone-aware TTL caching
PDF Ingestion	pdfplumber + pytesseract	latest	OCR and text chunk extraction
Language	Python	3.11	Entire system — zero framework deps

14 Bonus Features

Bonus	Description	File
A — Planning	Agent writes a 1-3 sentence execution plan before any tool call. Stored in trace JSON. Shows reasoning process, not just outputs.	agent.py → _build_plan()
B — Telemetry	Every trace includes per-tool latency (ms), call count, and failure count. Enables benchmarking and regression detection across runs.	utils/tracer.py
C — Reflection	After composing an answer, the LLM self-checks: Is the question answered? Is every claim cited? Contradictions? Failures trigger additional retrieval.	layers/reflection.py
D — Degradation	Full graceful degradation: 3-model ladder, force-router bypass, structured refusal. No hallucination observed under any failure mode during evaluation.	EVALUATION.md

15 Cost Analysis & Reproduction

Component	Provider	Plan	Cost
LLM — Llama 3.3-70b	Groq	Free tier	■0
Web Search	Tavily	Free tier (1,000/month)	■0
FAISS Index	Local	Local compute	■0
BM25	Local	Local compute	■0
SQLite	Local	Local compute	■0
TOTAL — full 20-Q eval	—	All free tiers	■0

Reproduction Steps

- 1. git clone https://github.com/Santhosh-2226/AGENTIC-RAG-PRODAPT
- 2. cd AGENTIC-RAG-PRODAPT && pip install -r requirements.txt
- 3. cp .env.example .env # Add GROQ_API_KEY and TAVILY_API_KEY
- 4. Place CSV files in data/raw/ and IPL PDF files in data/docs/
- 5. python ingest/ingest_data.py # builds ipl.db (179,372 rows)
- 6. python ingest/ingest_docs.py # builds FAISS + BM25 index (847 chunks)
- 7. python tools/query_data.py && python tools/search_docs.py && python tools/web_search.py
- 8. python main.py --eval --fresh # runs full 20-question evaluation
- 9. # Results written to eval/results.md and eval/results.json

Built by Santhosh · 2025 · AI & Data Science
github.com/Santhosh-2226/AGENTIC-RAG-PRODAPT